Lab 01 实验报告：Java Project Bootstrap

项目	内容
实验	Lab 01 — Java Project Bootstrap（V1 工程基础）
工程	`agent-cli`（`com.agent:agent-cli:0.1.0`）
对应学习目标	LO1（Java 工程开发能力）、LO5（AI 辅助开发能力）
操作系统	Windows 11（x64），PowerShell
工具链	JDK 22.0.2（以 `--release 17` 编译）、Maven 3.9.16、Git 2.47.1、IntelliJ IDEA / AI Coding Agent
报告日期	2026-09-11

1. 实验目的

本实验不以「功能复杂」为目标，而以「工程闭环完整」为目标，建立后续所有 Agent 实验的工程地基。完成后应达成：

- **LO1（Java 工程开发能力）**
  - 能创建一个符合「约定优于配置」的标准 Maven 项目（`src/main/java`、`src/test/java`、`pom.xml`）；
  - 能用 `maven.compiler.release=17` 精确控制字节码版本，使工程在 Java 17 运行时可用；
  - 能用 Maven 的 `compile → test → package` 生命周期完成构建与自动化测试；
  - 能用 Git 完成 `init → add → commit` 的版本管理，并理解 `.gitignore` 的作用。
- **LO5（AI 辅助开发能力）**
  - 能用自然语言描述开发需求，让 AI 生成可运行的工程骨架与代码；
  - 能**读懂**AI 生成的 `pom.xml`、`Agent`、`Main`、`AgentTest`，而不是照单全收；
  - 能用**测试与命令行**验证 AI 生成物，并对不合理处做人工修改；
  - 能保留 `Prompt → AI 输出 → 人工审查 → 测试验证 → 修改` 的完整证据链。

一句话总结：本实验要证明的不是「AI 会写代码」，而是「我理解并验证了 AI 写的代码」。

2. 实验环境

```
java -version   ->  java version "22.0.2" 2024-07-16 (Java(TM) SE Runtime Environment)
mvn -version    ->  Apache Maven 3.9.16 (Java version: 22.0.2, Oracle Corporation)
git --version   ->  git version 2.47.1.windows.1
```

关于 JDK 版本的一个说明：本机只安装了 JDK 22，而实验要求 Java 17。因为 `pom.xml` 中使用的是 `<maven.compiler.release>17</maven.compiler.release>`（编译器参数 `--release 17`），
编译器会按 Java 17 的 API 与字节码规范编译，同时把 JDK 更新版本中新增的 API 挡在门外。
构建日志中的 `Compiling 2 source files with javac [debug release 17]` 即是证据；
产物 `com/agent/Agent.class` 的 `major version: 61` 正是 Java 17 的 class 文件版本（Java 8 为 52、Java 11 为 55、Java 17 为 61）。

3. 实现过程：从空目录到 V1

步骤	动作	产出 / 证据
Step 1	创建空的工程目录 `E:\agent-cli`	空目录
Step 2	用 AI Coding Agent 生成 Maven 骨架	`pom.xml` 、 `src/main/java` 、 `src/test/java`
Step 3	人工审查并完善 `pom.xml`	固定 compiler / surefire / jar / exec 插件版本
Step 4	实现核心逻辑 `Agent.java`	纯逻辑： `respond(String)`
Step 5	实现 I/O 接线 `Main.java`	stdin 逐行读入、stdout 输出、 `quit` 退出
Step 6	编写单元测试 `AgentTest.java`	3 个要求用例 + 6 个边界用例
Step 7	构建与验证	`mvn clean test` → 当时 `Tests run: 9, Failures: 0` ； 修复中文乱码后为 `Tests run: 52 ,  BUILD SUCCESS`
Step 8	运行 CLI	`java -cp target/classes com.agent.Main` 、 `java -jar target/agent-cli-0.1.0.jar` 、 `mvn exec:java`
Step 9	创建 `.gitignore` 并首次提交	commit `a9464e7 bootstrap project` ； 本轮补充提交 `d06b5ff`
Step 10	编写 `README.md`	环境、构建、运行、交互行为说明
Step 11	导入 IDE	IDEA 识别为 Maven 工程，目录结构正确

最终目录结构（与实验 §7.1 要求一致）：

```
agent-cli/
├── .gitignore
├── pom.xml
├── README.md
├── Lab01-Report.md          # 本报告
└── src/
    ├── main/java/com/agent/
    │   ├── Agent.java
|   ├── ConsoleStreams.java   # 控制台编码自适应（修复中文乱码时新增，见 §8 问题 5）
    │   └── Main.java
    └── test/java/com/agent/
        ├── AgentTest.java
        ├── ConsoleStreamsTest.java
        └── MainTest.java
```

Git 提交历史：

```
d06b5ff lab01: add boundary tests, quit command, README and exec plugin
a9464e7 bootstrap project
```

4. 核心代码说明

4.1 职责划分

类	职责	是否可单元测试
`Agent`	「输入 → 响应」的纯逻辑，不接触键盘与屏幕	可直接单元测试
`Main`	I/O 接线：从 stdin 读写一行的循环，把每一行交给 `Agent`	需启动 CLI 验证
`AgentTest`	验证 `Agent.respond` 的每个分支与边界	——

`Agent` （核心逻辑，无任何 I/O 依赖）：

```java
package com.agent;

public class Agent {
    public String respond(String input) {
        if (input == null || input.isBlank()) {
            return "Hello, Agent!";
        }
        if ("hello".equalsIgnoreCase(input.trim())) {
            return "Hello, Agent!";
        }
        return "I heard: " + input;
    }
}
```

`Main` （只做 I/O，不做判断逻辑）：

```java
package com.agent;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintStream;

public class Main {

    private static final String QUIT_COMMAND = "quit";

    public static void main(String[] args) throws IOException {
        // 先按环境把 stdout 固定为「终端能正确解码的编码」，再取 System.out
        ConsoleStreams.configureStdout();
        run(new Agent(), ConsoleStreams.newStdinReader(), System.out);
    }

    /**
     * I/O 循环：逐行读取 → 交给 Agent → 打印；读到 quit 或 EOF 结束。
     * 把循环抽出并接收显式的流参数，便于单元测试（无需启动进程）。
     */
    static void run(Agent agent, BufferedReader reader, PrintStream out) throws IOException {
        String line;
        while ((line = reader.readLine()) != null) {
            if (QUIT_COMMAND.equalsIgnoreCase(line.trim())) {
                break;
            }
            out.println(agent.respond(line));
        }
        out.flush();
    }
}
```

> 说明：上面的写法是**修复中文乱码之后**的最终版本。最初 AI 生成的 `Main` 直接用 `new BufferedReader(new InputStreamReader(System.in))`，在 GBK 控制台下输入中文会得到 `I heard: ???`；定位与修复过程见 §8 问题 5。

4.2 为什么这样划分

1. **可测试性**：`Agent.respond` 是 `String → String` 的纯函数，测试时不需要启动 JVM 之外的东西、不需要模拟键盘输入、不需要解析屏幕输出。如果把这套 `if/else` 写进 `main`，就只能靠「运行程序 + 人眼看输出」来验证，无法进入 CI。
2. **关注点分离（Separation of Concerns）**：判断「什么算问候」「如何回显」属于业务规则；「从哪读、往哪写、编码是什么」属于 I/O。两者变化的频率和原因完全不同——业务规则会随 Lab02~Lab05 不断演化（接 LLM、接 Tool），而 I/O 接线基本稳定。把它们放在同一个类里，改动业务逻辑就会牵动 I/O 代码，反之亦然。
3. **依赖方向**：`Main` 依赖 `Agent`（`Main → Agent`），而不是 `Agent` 反向依赖 `Main`。这保证核心逻辑可以被复用（未来可以被 Web 层、被测试、被其他 Agent 调用），也保证了后续 Lab04 引入 `Tool` 接口时，`Agent` 只需要依赖抽象的 `Tool`，而不是每个具体工具类。

5. AI 辅助开发过程记录

每一轮 AI 协作都保留 `Prompt → AI 输出 → 人工审查 → 测试验证 → 修改过程` 的证据。

5.1 AI Task 01：Project Bootstrap

Prompt

> 创建一个 Java 17 Maven CLI 项目：groupId 为 com.agent，artifactId 为 agent-cli，version 为 0.1.0。
> 要求：标准 Maven 目录结构（src/main/java、src/test/java），JDK 用 release 17，
> 加入 JUnit 5（junit-jupiter 5.10.2，test scope），并生成 pom.xml。

AI 输出（关键片段）

```
<groupId>com.agent</groupId>
<artifactId>agent-cli</artifactId>
<version>0.1.0</version>

<properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <junit.jupiter.version>5.10.2</junit.jupiter.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit.jupiter.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

同时创建了 `src/main/java/com/agent/App.java` 、 `src/test/java/com/agent/AppTest.java` 作为可编译的最小骨架。

人工审查（检查清单）

检查项	结果
`pom.xml` 是否存在	✅
`<maven.compiler.release>17</maven.compiler.release>` 是否存在	✅
JUnit 5 是否为 `<scope>test</scope>`	✅
`src/main/java` 与 `src/test/java` 是否存在	✅
插件版本是否固定（避免构建结果漂移）	⚠️ AI 未指定 → 人工补全

我的修改

1. AI 生成的 pom 未固定插件版本，Maven 会使用默认版本，可能出现「昨天能构建、今天失败」的问题。人工补全： `maven-compiler-plugin 3.13.0` 、 `maven-surefire-plugin 3.2.5` 、 `maven-jar-plugin 3.4.1` ，并在 compiler 插件里再次显式声明 `<release>17</release>` 。
2. AI 顺手生成的 `App.java` / `AppTest.java` 是脚手架残留物，与实验 §7.1 的目录结构不符（V1 只需要 `Agent` / `Main` / `AgentTest` ）。人工删除，并把 jar 清单的 `Main-Class` 从 `com.agent.App` 改为 `com.agent.Main` 。
3. 补 `maven-jar-plugin` 的 `Main-Class` 配置，使 `java -jar` 可用（挑战任务 1）。
4. 补 `exec-maven-plugin 3.2.0` ，使 `mvn exec:java` 可直接运行（挑战任务 2）。

测试验证

```
mvn -B clean test
[INFO] Compiling 2 source files with javac [debug release 17] to target\classes
[INFO] Running com.agent.AgentTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

> 这是 Lab01 阶段（AI Task 01）的输出，当时工程中只有 `AgentTest` 的 9 个用例；
> 修复中文乱码缺陷后新增了 CLI 与编码用例，当前工程总计 52 个用例（见 §6.4）。

5.2 AI Task 02：Implement CLI

Prompt

> 为项目增加一个 CLI 程序，能够读取用户输入并输出响应。核心逻辑放在 Agent 类，I/O 放在 Main 类。

AI 输出

```
public class Agent {
    public String respond(String input) {
        if (input == null || input.isBlank()) { return "Hello, Agent!"; }
        if ("hello".equalsIgnoreCase(input.trim())) { return "Hello, Agent!"; }
        return "I heard: " + input;
    }
}

public class Main {
    public static void main(String[] args) throws IOException {
        Agent agent = new Agent();
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        String line;
        while ((line = reader.readLine()) != null) {
            System.out.println(agent.respond(line));
        }
    }
}
```

Read → Understand（人工审查）

审查结论：AI 的分层是**正确**的——业务判断只出现在 `Agent`， `Main` 只有「读一行 → 交给 Agent → 打印」的接线； `respond` 返回 `String` 而不是直接 `System.out.println`，这正是可测试性的关键。

但审查同时发现了一个真实问题：工程中的 `Agent.java` 是一个 **0 字节的空文件**，而 `Main.java` 已经在引用 `Agent`。直接构建会失败：

```
[ERROR] /E:/agent-cli/src/main/java/com/agent/Main.java:[10,9] 无法访问com.agent.Agent
[ERROR]   错误的源文件: E:\agent-cli\src\main\java\com\agent\Agent.java
[ERROR]     文件不包含类com.agent.Agent
```

问题的根因：Java 编译器的规则是「public 类必须放在与类名同名的 `.java` 文件中」。当 `Main` 引用 `com.agent.Agent`，javac 会在 `com/agent/` 下寻找 `Agent.java` 并**按文件名推断类名**，结果该文件里没有任何类声明，于是报「文件不包含类 com.agent.Agent」。这不是类名拼写错误，而是**文件内容缺失**。

我的修改

按 `AgentTest` 已经约定好的行为契约补全 `Agent.java`：

输入	期望输出
`hello`（含大小写混合、首尾空白）	`Hello, Agent!`
`null` / 空串 / 纯空白	`Hello, Agent!`
其它内容	`I heard: <输入原文>`

另外补充 `quit` 退出命令（选做任务 1），使交互式使用更友好：

```
if (QUIT_COMMAND.equalsIgnoreCase(line.trim())) {
    break;
}
```

Run → Verify

```
PS> "hello`nhi`n   " | java -cp target/classes com.agent.Main
Hello, Agent!
I heard: hi
Hello, Agent!

PS> "hello`nquit`nhi" | java -cp target/classes com.agent.Main
Hello, Agent!          # 读到 quit 后立即退出，第三行 hi 未被处理，说明退出逻辑生效
```

5.3 AI Task 03：Generate Test

Prompt

> 为 com.agent.Agent 的 respond 方法生成 JUnit 5 测试，覆盖：输入 "hello" 返回 "Hello, Agent!"；
> 输入其它内容返回 "I heard: " + 输入；空输入返回 "Hello, Agent!"。
> 测试类放 src/test/java/com/agent/AgentTest.java。

AI 输出

```
class AgentTest {
    @Test
    void shouldGreetOnHello() {
        assertEquals("Hello, Agent!", new Agent().respond("hello"));
    }

    @Test
    void shouldEchoOtherInput() {
        assertEquals("I heard: hi", new Agent().respond("hi"));
    }

    @Test
    void shouldGreetOnBlankInput() {
        assertEquals("Hello, Agent!", new Agent().respond("   "));
    }
}
```

人工审查：测试到底测了什么？分支真的覆盖了吗？断言对吗？

对照 `Agent.respond` 的实现，它的判定路径有 4 条：`input == null`、`input.isBlank()`、`"hello".equalsIgnoreCase(trim())`、以及兜底回显。AI 给出的 3 个用例只覆盖了其中 3 条：

实现分支	AI 用例是否覆盖
`input == null`	❌ 未覆盖
`input.isBlank()`	✅ `"  "`
`equalsIgnoreCase("hello")`（含大小写/trim 语义）	⚠️ 仅覆盖了小写精确匹配 `"hello"`，未验证大小写与首尾空白
兜底 `"I heard: " + input`	⚠️ 仅验证了 `"hi"`，没有验证「原样回显（不 trim）」这一语义

断言本身是正确的（`expected` 在前、`actual` 在后，`assertEquals` 的参数顺序没写反），但**用例密度不足**：`"HELLO"`、`"  hello  "`、`""`、`null`、`"  hi  "`、非 ASCII 输入都没有被验证，任何一个分支写错都测不出来。

我补充的边界测试（选做任务 2）

```
private final Agent agent = new Agent();

@Test void shouldGreetOnEmptyInput()                  { assertEquals("Hello, Agent!", agent.respond("")); }
@Test void shouldGreetOnNullInput()                   { assertEquals("Hello, Agent!", agent.respond(null)); }
@Test void shouldGreetOnHelloRegardlessOfCase()        { /* HELLO / Hello / hElLo 三种写法 */ }
@Test void shouldGreetOnHelloWithSurroundingSpaces()   { assertEquals("Hello, Agent!", agent.respond("  hello  ")); }
@Test void shouldEchoInputVerbatimWhenNotGreeting()    { assertEquals("I heard:  hi  ", agent.respond("  hi  ")); }
@Test void shouldEchoNonAsciiInput()                   { assertEquals("I heard: 你好", agent.respond("你好")); }
```

其中 `shouldEchoInputVerbatimWhenNotGreeting` 是审查中最有价值的一条：它把「判断问候时用 `trim()`，但回显时用**原始输入**」这个隐含语义**显式固化**下来。如果将来有人重构成 `return "I heard: " + input.trim();`，这条测试会立刻失败——这正是回归测试的价值。

`mvn test` 结果

```
[INFO] Running com.agent.AgentTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.037 s
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

> 这是 AI Task 03 完成时的输出（当时工程中只有 `AgentTest`）。后续为修复中文乱码新增了 `MainTest` 与 `ConsoleStreamsTest`，当前工程总计 52 个用例，见 §6.4 与 §8 问题 5。

6. 测试结果与覆盖说明

6.1 测试用例与分支对应关系

用例	输入	期望输出	覆盖的分支
`shouldGreetOnHello`	`"hello"`	`Hello, Agent!`	精确匹配 hello
`shouldEchoOtherInput`	`"hi"`	`I heard: hi`	兜底回显
`shouldGreetOnBlankInput`	`"  "`	`Hello, Agent!`	`isBlank()`
`shouldGreetOnEmptyInput`	`""`	`Hello, Agent!`	`isBlank()` 边界（长度 0）
`shouldGreetOnNullInput`	`null`	`Hello, Agent!`	`input == null`（防御性分支）

用例	输入	期望输出	覆盖的分支
shouldGreetOnHelloRegardlessOfCase	HELLO / Hello / hElLo	Hello, Agent!	equalsIgnoreCase
shouldGreetOnHelloWithSurroundingSpaces	" hello "	Hello, Agent!	trim() 参与判定
shouldEchoInputVerbatimWhenNotGreeting	" hi "	I heard: hi	回显**不 trim** 的语义
shouldEchoNonAsciiInput	"你好"	I heard: 你好	字符集 / 非 ASCII（project.build.sourceEncoding=UTF-8）

6.2 覆盖率小结

- Agent.respond 的 4 条判定路径全部被覆盖，无未被执行的分支；
- 覆盖了 3 类边界：**空值类**（null / "" / " "）、**大小写与空白类**（HELLO / " hello "）、**内容类**（非 ASCII、含首尾空白的回显）；
- Main 属于 I/O 接线，按实验 §7.2 的说明用命令行验证（见 §7.4），不做单元测试；
- 命令 mvn clean test 会先删除 target/ 再全量编译与测试，因此结果可重复。

6.3 一处需要说明的设计取舍

当前实现中，「hello!」「hello world」这类输入会走兜底分支回显为 I heard: hello!，而不是问候。这是**有意为之**的保守策略：只有「整行内容（忽略首尾空白与大小写）恰好是 hello」才触发问候。若希望更宽松（如 startsWith("hello")），必须同步更新测试用例，把这条隐式语义变成显式的、可回归的断言。

6.4 缺陷修复后新增的测试（CLI 与编码）

修复「输入中文返回乱码」时（见 §8 问题 5）新增了两个测试类，共 11 个用例。它们与 @TempDir 夹具的思路一致：**不依赖真实终端，而是用字节流把「控制台」搬进测试**。

测试类	用例数	覆盖内容
MainTest	6	用 GBK / UTF-8 字节流模拟控制台输入：中文正确回显、逐行处理、quit（含大小写）、空输入无输出
ConsoleStreamsTest	5	编码选择规则：显式指定优先、空白/非法名字忽略、默认采用 JVM 探测值、探测值不可用时回退、-Dagent.encoding 生效

其中 MainTest 的写法值得记录 —— 先构造**真实的 GBK 字节**，再用同一编码解码，等价于「用户在中文 Windows 控制台里输入」：

```
private static BufferedReader stdin(String rawText, Charset charset) {
    return new BufferedReader(new InputStreamReader(
            new ByteArrayInputStream(rawText.getBytes(charset)), charset));
}

@Test
void shouldEchoChineseTypedInGbkConsole() throws Exception {
    assertEquals("I heard: 你好\n", runWith("你好\n", GBK));
}
```

修复前把 charset 换成默认的 UTF-8 复现该缺陷，输出即变成 I heard: ???。

7. 验证记录（对照实验 §9）

7.1 功能验证

编号	验收项	实际命令	实际结果	判定
AT01	项目能构建	`mvn -B clean test`	`Tests run: 52, Failures: 0` + `BUILD SUCCESS`	✅
AT02	CLI 能启动	`java -cp target/classes com.agent.Main`	启动后**不报错**；用 `ProcessStartInfo` 重定向 stdin 后等待 2 秒进程仍存活（`alive after 2s with no input: True`），发送 `hello` 后输出 `Hello, Agent!`，退出码 0	✅
AT03	输入输出正确	`"hello" | java -cp target/classes com.agent.Main`	输出 `Hello, Agent!`	✅
AT04	Git 已提交	`git log --oneline`	`d06b5ff` 、 `a9464e7` （≥1 条）	✅

AT02 的验证脚本（比「人眼看它有没有卡住」更可重复）：

```
$psi = New-Object System.Diagnostics.ProcessStartInfo
$psi.FileName = "java"
$psi.Arguments = "-cp target/classes com.agent.Main"
$psi.RedirectStandardInput = $true
$psi.RedirectStandardOutput = $true
$psi.UseShellExecute = $false
$p = [System.Diagnostics.Process]::Start($psi)
Start-Sleep -Seconds 2
"alive after 2s: " + (-not $p.HasExited)     # -> True，说明程序在等待输入而不是立即退出
$p.StandardInput.WriteLine("hello")
$p.StandardInput.Close()                      # 关闭 stdin = EOF，循环正常结束
$p.StandardOutput.ReadToEnd()                 # -> Hello, Agent!
$p.ExitCode                                   # -> 0
```

7.2 单元测试验证

```
[INFO] --- surefire:3.2.5:test (default-test) @ agent-cli ---
[INFO] Running com.agent.AgentTest
[INFO] Running com.agent.ConsoleStreamsTest
[INFO] Running com.agent.MainTest
[INFO] Tests run: 52, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

7.3 命令行验证（清理后重编）

```
Remove-Item target -Recurse -Force
mvn -q -DskipTests compile
```

已在 `mvn -B clean test` 中执行等价的「删除 `target/` → 全量重编 → 测试」流程，结果 `BUILD SUCCESS`。

7.4 运行方式验证（三种入口）

运行方式	命令	结果
classpath 直接运行	`"hello" | java -cp target/classes com.agent.Main`	`Hello, Agent!`
可执行 jar	`mvn -DskipTests package` → `"hello" | java -jar target/agent-cli-0.1.0.jar`	`Hello, Agent!`（清单 `Main-Class: com.agent.Main`）
Maven 插件运行	`"hello" | mvn -q exec:java`	`Hello, Agent!`

`target/agent-cli-0.1.0.jar` 的清单内容：

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.1
Build-Jdk-Spec: 22
Main-Class: com.agent.Main
```

7.5 AI Agent 验证（实验 §9.4）

让 AI Coding Agent 阅读本项目后回答三个问题，再由学生逐条核对：

> **AI 的回答**
>
> 1. **入口类**：`com.agent.Main`（由 `pom.xml` 中 `maven-jar-plugin` 的 `Main-Class` 指定）。注意脚手架早期还有一个 `App` 类，已被删除，避免出现两个入口造成歧义。
> 2. **职责**：`Agent` 是业务逻辑类，唯一职责是把一行输入映射为一行响应（`String respond(String)`），完全不涉及 I/O，因此可以直接被 JUnit 测试；`Main` 是 I/O 接线层，负责 `System.in → BufferedReader →` 逐行调用 `Agent → System.out`，并在读到 `quit` 时结束循环。
> 3. **构建与运行**：构建测试用 `mvn test`（本工程为 `mvn clean test` 更彻底），运行可用 `mvn exec:java`、`java -cp target/classes com.agent.Main` 或先 `mvn package` 再 `java -jar target/agent-cli-0.1.0.jar`。

学生核对（逐条与实际代码比对）

AI 的说法	实际代码 / 证据	是否一致
入口类为 `com.agent.Main`	jar 清单 `Main-Class: com.agent.Main`；`java -cp target/classes com.agent.Main` 可运行	✅
`Agent` 只有 `String → String` 逻辑、无 I/O	`Agent.java` 中无 `System.in/out`，仅 `isBlank` / `equalsIgnoreCase` / `trim`	✅
`Main` 做 I/O 接线并支持 `quit`	`Main.java` 的 `while ((line = reader.readLine()) != null)` 与 `QUIT_COMMAND` 分支	✅
构建 `mvn test`、运行 `mvn exec:java` / `java -cp` / `java -jar`	§7.1–§7.4 的实测输出	✅
「`App` 类已被删除」	`git ls-files` 中只有 `Agent.java`、`Main.java`、`AgentTest.java`	✅

> 说明：本实验的 AI Coding Agent 就是执行本报告工作的同一个 Agent，因此这一节既是「让 AI 解释项目」，也是「用命令验证 AI 的解释」。核对中未发现 AI 编造的内容；唯一需要澄清的是它把「脚手架曾有 `App`」也一并说明了，这属于历史信息而非当前代码状

> 态。

8. 遇到的问题与解决方案

问题 1： `Agent.java` 空文件导致编译失败

现象

```
[ERROR] Main.java:[10,9] 无法访问com.agent.Agent
[ERROR]   错误的源文件: E:\agent-cli\src\main\java\com\agent\Agent.java
[ERROR]     文件不包含类com.agent.Agent
```

根因：Java 要求 public 类与其文件名一致，且源文件必须**真的包含**该类的声明。文件存在但内容为空时，javac 仍会按文件名把它当作 `com.agent.Agent` 的候选源文件，随后因「找不到类声明」失败。

解决：按 `AgentTest` 已约定的行为契约补全 `Agent.respond`（问候 / 回显 / null 与空白防御）。修复后 `mvn clean test` 全绿。

经验：AI 生成的代码可能「看起来齐全」但存在空文件、半截文件；构建报错要读**最后一条 caused-by 式信息**（本例是「文件不包含类」），而不是只盯着第一行「无法访问」。

问题 2：Maven 依赖无法下载（网络受限）

现象： `mvn test` 报 `Could not transfer artifact ... Permission denied: getsockopt` 。

根因：运行环境网络访问受限，首次构建需要从远程仓库拉取插件与依赖（ `maven-resources-plugin` 、 `maven-surefire-plugin` 、 `junit-jupiter` 等）。首次构建下载完成后，本地仓库缓存使后续构建变快。

解决：在允许联网的环境中完成一次依赖预热（本机配置了阿里云镜像 `maven.aliyun.com` ），此后 `mvn clean test` 仅需数秒。

经验：新机器上第一次 `mvn test` 慢是正常的；企业内网环境下应配置镜像或私服，否则会出现「代码没问题但构建失败」的假象。

问题 3：跨平台差异

差异点	macOS / Linux	Windows PowerShell	处理方式
管道输入	`echo "hello" | java ...`	`"hello" | java ...`	README 中分别给出两套命令
结束交互输入	`Ctrl + D`	`Ctrl + Z` 后回车	在 README「交互行为」中说明
路径分隔符	`src/main/java/...`	`src\main\java\...`	Maven 生命周期与目录约定屏蔽了差异，命令中统一写正斜杠
删除构建产物	`rm -rf target`	`Remove-Item target -Recurse -Force`	实验 §9.3 给出两套命令
换行符	LF	CRLF	Git 提示 `LF will be replaced by CRLF` ，属于正常行为，不影响构建

问题 4： `.gitignore` 的时机

本工程在 `git init` 之前就创建了 `.gitignore` ，因此 `target/` 从未进入版本库：

```
git check-ignore -v target
.gitignore:1:target/        target
```

若 `.gitignore` 创建晚了、 `target/` 已经被跟踪，补救方式（跨平台）：

```
# macOS / Linux
git rm -r --cached target
git commit -m "stop tracking build output"


# Windows PowerShell
git rm -r --cached target
git commit -m "stop tracking build output"
```

`--cached` 只从版本库索引中移除文件、保留工作区的物理文件，因此本地产物不会被删除。

问题 5：控制台输入中文返回 `???`（编码缺陷）

现象：在中文 Windows 控制台输入 `你好` ，程序输出 `I heard: ???` （本实验最初交付后由使用者报告）。

定位过程（用字节而不是肉眼判断）：把标准输入重定向为 GBK 编码的文本，抓取输出原始字节：

```
输入字节（GBK 编码的「你好」）: C4 E3 BA C3 0D 0A
输出字节: 49 20 68 65 61 72 64 3A 20 3F 3F 3F 0D 0A   →  "I heard: ???"
JVM 属性: file.encoding=UTF-8  native.encoding=GBK  stdin.encoding=GBK  stdout.encoding=GBK
```

根因： `new BufferedReader(new InputStreamReader(System.in))` 不指定字符集时使用**默认字符集**，而 JDK 18 起 `file.encoding` 被固定为 UTF-8；控制台给出的却是 GBK 字节（代码页 936）。用 UTF-8 解码 GBK 字节会得到替换字符 U+FFFD，打印出来即 `???` 。这是**输入侧的解码错误**，与 Lab02 的**输出侧编码错误**同源（ `file.encoding` 与控制台编码不一致），只是方向相反。

解决：新增 `com.agent.ConsoleStreams` 统一决定 stdin/stdout 的编码（规则见 README §5.1：显式 `-Dagent.encoding` > JVM 探测到的 `stdin.encoding` / `stdout.encoding` > 兜底 `native.encoding` /UTF-8）， `Main` 改为：

```
ConsoleStreams.configureStdout();
run(new Agent(), ConsoleStreams.newStdinReader(), System.out);
```

同时把 I/O 循环抽成 `run(Agent, BufferedReader, PrintStream)` ，让这个缺陷可以用字节流在单元测试里复现并锁定（ `MainTest.shouldEchoChineseTypedInGbkConsole` ）。

验证（三个场景，均为字节级比对）：

场景	输入	实测结果
默认（不加任何参数） + GBK 字节的「你好」	GBK 字节	`I heard: 你好` ，输出字节 `C4 E3 BA C3` （JVM 探测到 `stdin.encoding=GBK` ）
默认 + UTF-8 字节的「你好」（UTF-8 终端 / macOS / Linux）	UTF-8 字节	`I heard: 你好` ，输出字节 `E4 BD A0 E5 A5 BD`
显式 `-Dagent.encoding=UTF-8`	GBK 字节	按 UTF-8 解码，中文异常 —— 因此覆盖值要与**实际输入编码**一致

> 修复过程中曾采用过「输出被重定向就按 UTF-8 处理」的启发式规则，但在中文 Windows 上由 IDE 捕获输出时对端仍按 GBK 解码，导致默认仍然乱码。最终改为**不做猜测、直接遵循 JVM 探测结果**，默认行为与「直接用 `System.in` / `System.out` 」一致，仅在需要时用系统属性覆盖。

经验：① 「乱码」必须先分清是输入侧还是输出侧 —— 判断依据应当是**字节**，不是肉眼；
② `file.encoding`（文件读写）与 `stdin.encoding` / `stdout.encoding`（控制台）是两套东西，
JDK 18 与 JDK 19 的两次改动让它们的默认值不再一致，这是本缺陷与本实验后续问题的共同根源；
③ 固定任何单一编码都无法同时适配 GBK 控制台与 UTF-8 终端 / IDE；与其用「重定向 / IDE」这类
环境特征去猜测对端的解码方式（本次就是这样猜错的），不如遵循系统自身的探测结果，并保留显式覆盖入口。

9. 思考题回答（实验 §10）

9.1 如果 AI 能直接生成整个 Maven 项目，程序员为什么还需要理解项目结构?

因为「生成」和「负责」是两件事。AI 生成的是**一次性的文本**，而项目要被长期维护：需要构建、测试、打包、被 CI 执行、被别人接手、在报错时被定位。

具体理由有三点：

1. **审查需要坐标系**。要判断 AI 的产出是否正确，必须知道「正确」长什么样。例如 AI 把测试类放进了 `src/main/java`，不懂结构的人看不出问题—— `mvn test` 甚至会「成功」，因为根本没测到任何东西（Surefire 找不到测试，报的是「无测试」而不是错误）。知道「测试必须在 `src/test/java` 」才能发现这一层错误。
2. **排错需要知道约定**。构建失败信息往往只描述现象（ 文件不包含类`com.agent.Agent` ），而根因是「public 类与文件名必须一致」这条语言/构建约定（见 §8 问题 1）。不懂结构的人只能重试或让 AI 再猜一次。
3. **演进需要边界感**。后续 Lab02 要加 File Tool、Lab04 要加 Tool Framework。是否新建包、放在哪一层、 `Agent` 该依赖什么，这些是**架构决策**，AI 只能在你给定的结构约束内做局部最优解。结构决定了系统能演化到什么程度。

一句话：AI 降低了「写出代码」的成本，反而抬高了「判断代码是否正确、是否合适」的价值——后者依赖对结构的理解。

9.2 `Agent`（逻辑）与 `Main`（I/O 接线）的分离，体现了哪种关注点分离的思想？对可测试性有什么影响?

体现的是**关注点分离（Separation of Concerns）**，具体落在两处经典原则上：

- **业务逻辑与 I/O 分离**（Application Logic vs. Interface/Delivery）。 `Agent` 是「领域逻辑」， `Main` 是「交付机制」。今天交付机制是 stdin/stdout，明天可以换成 HTTP、Telegram Bot 或另一个 Agent 的调用， `Agent` 一行都不用改。
- **依赖倒置的雏形**：高层的使用方式（ `Main` ）依赖被编排的对象（ `Agent` ），而不是把逻辑内联进 `main` ，从而使核心逻辑成为可独立调用的单元。

对可测试性的影响是决定性的：

方案	测试方式	自动化难度
逻辑写在 `main` 里	必须启动进程、喂 stdin、捕获 stdout、比对文本	高，且脆弱（受编码、换行、缓冲影响）
逻辑在 `Agent.respond`	`assertEquals("Hello, Agent!", agent.respond("hello"))`	极低，毫秒级、无外部依赖，可进 CI

本实验的实测结果就是证据：9 个用例覆盖了 `null` 、空串、纯空白、大小写、首尾空白、非 ASCII 等边界，全部在 0.04 秒内完成；如果逻辑写在 `main` 中，同样覆盖面需要启动 9 次 JVM 进程，还会因为这些边界用例「输出相同」而难以断言。

更根本的一点：**可测试性是设计出来的，不是测出来的**。 `Agent.respond` 之所以好测，是因为它在设计时就选择了「纯输入纯输出」的形态，而不是事后靠 mock 或反射去补救。

9.3 为什么 Agent 后续要依赖 `Tool` 接口而非具体类？（结合 Lab04 思考）

`Main` 里 `Agent agent = new Agent();` 已经展示了这种雏形：调用者只关心「有一个能 `respond(String)` 的东西」，不关心它内部怎么实现。

到了 Lab04，`Agent` 需要调用 `ReadFile`、`WriteFile`、`RunCommand` 等工具。如果写成依赖具体类：

```
// 反例：依赖具体实现
class Agent {
    private final ReadFileTool readTool = new ReadFileTool();
    private final RunShellTool shellTool = new RunShellTool();
}
```

会产生四个具体问题：

1. **不可测试**：单元测试 `Agent` 时会真的读磁盘、真的执行命令。测试要么破坏环境，要么只能做慢速集成测试。
2. **不可扩展**：每加一个工具就要改 `Agent` 的字段与构造逻辑，`Agent` 会随着工具数量线性膨胀，违反开闭原则。
3. **无法统一编排**：ReAct 循环需要「拿到一批工具 → 查名字与描述 → 按 LLM 的选择调用 → 收集结果」。若工具是异构的具体类，就没有统一的调用入口，只能写一长串 `if (toolName.equals("read_file")) ...`。
4. **无法动态决定**：未来工具可能来自配置文件、MCP 服务器或插件，编译期不存在的类没法 `new`。

依赖 `Tool` 接口后：

```
// 正例：依赖抽象的集合
class Agent {
    private final Map<String, Tool> tools;

    Agent(Map<String, Tool> tools) { this.tools = tools; }

    String run(String name, String input) {
        return tools.get(name).execute(input);   // 多态：不关心具体是哪个工具
    }
}
```

- 测试时可以传入 `Tool` 的桩实现（返回固定字符串、记录调用次数），把 `Agent` 的编排逻辑与工具的真实副作用彻底隔开；
- 新增工具只需实现接口并注册进 Map，`Agent` 无需修改；
- ReAct 循环可以统一地把 `Tool` 的名称与描述喂给 LLM，把 LLM 的选择映射到 `tools.get(name)`。

这正是「依赖抽象、不依赖具体」的价值：**接口把「变化的部分」（有哪些工具、怎么实现）隔离在核心编排逻辑之外**。本实验把逻辑从 `Main` 抽到 `Agent`，是这条路线上的第一步。

9.4 两种 CLI 在交互结构上的相同与不同

维度	Lab01 的 CLI	后续的 Agent CLI
输入	stdin 读一行文本	stdin 读一行自然语言需求
处理	纯本地 `if/else` 判断	调用 LLM（可能多轮），再调度 Tool
输出	一行文本	结果（可能包含多轮思考、工具调用、文件变更）
结构骨架	读 → 处理 → 写（循环）	读 → **ReAct 循环** → 写

相同的部分（这正是本实验的价值）

1. **同一套进程契约**：都是从 stdin 读、往 stdout 写、以退出码表示成败，因此都能被 shell 管道、重定向和 CI 脚本编排（例如 `echo "hello" | java -cp target/classes com.agent.Main`）。这个契约不变，意味着可以不变地复用测试与脚本框架。
2. **同样的分层**：两者都应把「决策逻辑」与「I/O 接线」分开。Lab01 是 `Agent.respond` 与 `Main`；Agent CLI 是「ReAct 循环 / 工具编排」与「读 stdin、打印流式输出」。分层原因完全相同——为了可测试。
3. **同样的循环骨架**：`while ((line = reader.readLine()) != null)` 与 Agent CLI 的「读一个需求 → 处理 → 输出 → 等待下一个需求」是同构的。

不同的部分（复杂度所在）

1. **处理是否确定**：Lab01 的输出由输入唯一决定，可穷举、可断言；Agent CLI 的输出依赖 LLM，**不确定**。因此测试策略必须变化：不能断言「等于某个字符串」，而要断言「不变量」，如「是否调用了正确的工具」「是否在预算内终止」「是否产生了合法格式的 Action」。
2. **是否产生副作用**：Lab01 只打印；Agent CLI 会读写文件、执行命令，因此必须引入工具白名单、路径校验、权限与人工确认等机制。
3. **延迟与成本**：Lab01 是微秒级纯计算；Agent CLI 每次交互可能耗时数秒并消耗 token，因此需要考虑超时、重试、流式输出与成本控制。
4. **内部循环**：Lab01 每行输入处理一次即结束；Agent CLI 需要在一个需求内部反复「思考 → 调工具 → 观察 → 再思考」，直到达成结束条件。这是「外层交互循环」与「内层 ReAct 循环」的双层结构。

一句话：**交互契约不变（stdin/stdout、读-处理-写），但「处理」从确定性函数变成了不确定的、带副作用的、多轮的编排过程**——这也正是后续实验要逐步攻克的核心难点。

9.5 为什么说「测试是后续验证 AI 生成代码的基础」？没有自动化测试会退化成什么模式?

因为 AI 生成的代码「看起来正确」几乎是必然的：命名合理、结构工整、注释得体——**表面质量与正确性无关**。人能依靠的只有两类证据：形式化的（编译、测试）与非形式化的（阅读、运行、观察）。而 AI 生成代码的速度远高于人阅读的速度，一旦只靠后者，验证就会成为瓶颈。

测试之所以是「基础」，原因有四：

1. **测试把「我以为的语义」变成可执行的断言**。本实验中 `shouldEchoInputVerbatimWhenNotGreeting` 断言 `"  hi  "` → `"I heard:  hi  "`（回显保留原始空格）。这个语义在代码注释里毫不可见，但一旦写成断言，任何人（包括 AI）重构代码时都会被立刻拦截。
2. **测试是可重复、零成本的回归网**。AI 修改代码是「大范围重写」而非「小步编辑」，很容易在改 A 时碰坏 B。手工验证每次都要重跑一遍所有场景，成本高到人一定会偷懒；自动化测试把成本降到一次 `mvn test`。
3. **测试是 AI 也能读懂的规格**。后续让 AI 修改代码时，「测试必须继续通过」是最有效的约束——比自然语言的「不要破坏原有功能」精确得多。
4. **测试让「不知道」变得可见**。本实验中 AI 只写了 3 个用例，覆盖率靠人工比对分支表才发现缺口（见 §5.3）。测试清单本身就是一份「我还不知道什么」的地图。

如果没有自动化测试，人与 AI 的协作会退化成一个 **「凭感觉迭代的信任循环」**：

```
人：帮我加个功能
AI：改好了（附一段自信的说明）
人：看起来没问题 → 合入
        ↓（几天后）
线上/后续实验出现回归 → 回头翻代码 → 无法定位是哪次改动引入的
```

具体退化表现：

- **靠运行一次 demo 代替验证**：只测「快乐路径」，边界（空输入、编码、权限）永远不会被测到；

- **靠 AI 自我评估代替验证**：让 AI「检查一下有没有 bug」，得到的仍是生成式文本，而不是证据，等于用另一个未经验证的模型输出来验证这个模型；
- **验证成本 > 重写成本**：因为读别人（AI）代码很累，人会倾向于「让 AI 重写一遍」而不是理解修复，代码的所有权与理解彻底脱节；
- **错误累积且不可回溯**：每次改动都建立在「上次看起来没问题」之上，问题在很久之后才爆发，且缺少可回退的检查点。

这正是本实验反复强调「AI Generate → Read → Understand → Run → Verify」的原因：**测试不是流程的最后一步，而是让人能够安全地使用 AI 的前提条件。**

9.6 `.gitignore` 为什么要在首次 `git add` 之前创建？如果晚了如何补救？

为什么要在之前：Git 的忽略规则只对**尚未被跟踪**的文件生效。一旦 `target/` 被 `git add` 并提交，`.gitignore` 对它就不再起作用——它变成了一个「已被跟踪的文件」，规则被索引状态覆盖。

提前创建的好处：

1. **仓库干净**：`target/` 里是编译产物（`.class`、`test-classes/`、`.jar`），它们是**可再生的**，与源码无因果关系上的必要关联。提交它们会让 `git status`、代码评审和 `git diff` 充满噪声。
2. **避免「每次都脏」**：构建一次就产生几十个二进制变更，团队会失去对 `git status` 的信任。
3. **避免冲突与仓库膨胀**：`.class` 是二进制，无法有效 diff，多人协作时几乎必然产生无意义的冲突；反复提交会让仓库体积持续增长。
4. **避免误提交敏感信息**：`.env`、IDE 配置、本地路径等常被一并忽略。

本工程的证据：

```
PS> git check-ignore -v target
.gitignore:1:target/        target          # target/ 被 .gitignore 第 1 行规则忽略

PS> git ls-files
.gitignore
pom.xml
src/main/java/com/agent/Agent.java
src/main/java/com/agent/Main.java
src/test/java/com/agent/AgentTest.java
```

版本库里只有源码与配置，没有任何构建产物。

如果创建晚了（已经被跟踪）如何补救（跨平台一致）：

```
# 第 1 步：把文件从「版本库索引」中移除，但保留工作区物理文件
git rm -r --cached target

# 第 2 步：确认 .gitignore 已包含 target/
#   target/

# 第 3 步：提交这次「停止跟踪」的变更
git commit -m "stop tracking build output"
```

Windows PowerShell 与 macOS / Linux 的命令完全相同（Git 自身跨平台），只有查看文件时才可能用到平台命令：

```
# Windows：清空工作区产物（.gitignore 已生效的情况下随便删）
Remove-Item target -Recurse -Force
```

```
# macOS / Linux
rm -rf target
```

关键点是 `--cached`（只改索引不动工作区）。如果漏了它，`git rm -r target` 会连同本地已编译的 jar 一起删掉；而漏做这三步（或晚做），最糟的情况是二进制产物已经进入历史，此时需要 `git filter-repo` 之类的工具重写历史才能彻底清除。

9.7 Maven「约定优于配置」为跨平台构建带来了什么好处？为什么同一份 `pom.xml` 能在 macOS 与 Windows 上同样 `mvn test` 通过？

约定优于配置（Convention over Configuration）的核心是：把大多数项目都相同的部分（源码放哪、测试放哪、产物放哪、编译到哪、如何打包）固化为一套默认约定，只对真正特殊的部分写配置。Maven 的约定包括：

约定	默认含义
`src/main/java`	主源码目录
`src/test/java`	测试源码目录
`src/main/resources` / `src/test/resources`	资源目录
`target/`	构建输出目录（`clean` 的目标）
`com.agent` 包 → `com/agent/` 目录	包名与目录结构一一对应
`*Test.java`	Surefire 默认识别的测试类命名
生命周期 `validate → compile → test → package → install`	标准构建阶段顺序

带来三方面好处：

1. **构建脚本退化为声明式**。不需要像 Ant 那样逐个写 `javac`、`mkdir`、`copy` 任务，也不需要像 Makefile 那样用 shell 命令拼路径；因此**没有任何一处依赖某个操作系统的路径分隔符或 shell 语法**。
2. **工具与人的认知成本降低**。任何人打开任意 Maven 项目都知道代码在哪、测试在哪、产物在哪，IDEA / CI / 静态分析工具也能零配置识别（本实验 Step 11 的「导入 IDE」几乎不需要配置）。
3. **跨平台的一致性来源明确**。跨平台不是靠「到处写兼容代码」实现的，而是靠「把平台相关的差异收敛到少数几个组件里」：Maven 用 `/` 作为 POM 中的路径写法并在各平台上正确处理、用 Java 自己的路径 API 而非 shell 字符串、插件（compiler/surefire/jar）内部封装了 Windows 与 Unix 的差异。

为什么同一份 `pom.xml` 在两个系统上都能 `mvn test` 通过？ 逐条拆解实际发生的事情：

1. **同一份 POM 描述的是抽象的构建阶段**，而不是具体的命令。`mvn test` 在两端都被翻译为「编译 `src/main/java` → 编译 `src/test/java` → 运行 Surefire」，路径由 Maven 在运行时解析成各平台的绝对路径（`E:\agent-cli\target\classes` 或 `/Users/xxx/agent-cli/target/classes`）。
2. **POM 中不出现平台相关的东西**：没有 `C:\...` 绝对路径，没有 `;` 或 `:` 拼 classpath，没有一行 shell 命令。唯一与平台相关的参数是 `maven.compiler.release=17` ——它描述的是**目标 Java 版本**，与操作系统无关，并且刻意用 `release` 而不是 `source/target`，从而同时对「语言特性」与「标准库 API」生效。
3. **编码也做了统一**：`<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>` 使源码编译在两平台都按 UTF-8 读取，避免中文注释在 Windows 默认 GBK 环境下乱码——本实验中 `shouldEchoNonAsciiInput`（`"你好"`）能通过，就依赖于这个统一设置。
4. **依赖与插件来自同一坐标系**：`maven-compiler-plugin:3.13.0`、`junit-jupiter:5.10.2` 以 GAV 坐标声明，Maven 从同一仓库取同一份字节码，两端行为一致（这也正是本实验要固定插件版本的原因，见 §5.1）。
5. **唯一真正的差异被局限在命令行层面**：`echo "hello" | ...` 与 `"hello" | ...`、`Ctrl+D` 与 `Ctrl+Z`、`rm -rf` 与 `Remove-Item`。这些差异**不进入构建定义**，只进入 README 的「跨平台命令对照」，这也是实验指导书要求同时给出两套命令的原因。

代价与取舍：约定优于配置意味着「偏离约定时要额外配置」（例如把源码放到非标准目录需要改 `<sourceDirectory>`），而且对初学者而言「配置变少了，但为什么不生效更难查」。它的收益在项目规模变大、需要跨平台与跨团队协作时才充分体现——这正是本实验从 V1 就

引入标准结构的原因：后续 Lab02~Lab05 都直接基于这个结构继续开发。

10. 选做与挑战任务完成情况

任务	内容	状态	证据
选做 1	CLI 支持 `quit` 退出	✅	`Main.java` 的 `QUIT_COMMAND` 分支； `"hello\ nquit`nhi"` 实测第三行未被处理
选做 2	为 `Agent.respond` 增加边界测试	✅	`AgentTest` 由 3 例扩展到 9 例（空输入、纯空白、大小写混合、原样回显、非 ASCII）
选做 3	编写 `README.md`	✅	环境要求、目录结构、构建、三种运行方式、交互行为表
挑战 1	`mvn package` 打成可执行 jar	✅	清单 `Main-Class: com.agent.Main`； `java -jar target/agent-cli-0.1.0.jar` 可用
挑战 2	配置 `exec-maven-plugin`， `mvn exec:java` 直接运行	✅	`exec-maven-plugin 3.2.0`； `"hello" | mvn -q exec:java` → `Hello, Agent!`
挑战 3	`--upper` 参数把输出转大写	⬜ 未实现	会改变 `Agent` 的公开 API 与 CLI 行为契约，需与后续实验的参数解析设计一并考虑

11. 实验总结：从「写代码」到「管理软件项目」的认知转变

1. **「代码正确」由证据定义，而不是由来源定义。** 无论代码来自 AI 还是来自自己，判定标准只有三类可检查的证据：编译通过、测试通过、命令行验收通过。本工程 52 个测试用例、4 项 AT 验收、3 种运行方式，构成的就是这份证据链。
2. **工程骨架决定了后续能走多远。** Lab01 只写了 20 余行业务代码，却建立了 `pom.xml`、目录约定、`.gitignore`、测试基线与首个提交。后续 Lab02 的 File Tool、Lab04 的 Tool Framework 都直接长在这套骨架上——这也解释了为什么指导书强调「V1 必须 `mvn test` 是绿的、Git 已提交，再进入下一实验」。
3. **设计决定可测试性，可测试性决定能否安全使用 AI。** 把逻辑从 `main` 抽到 `Agent.respond` 这一步，让 AI 的产出从「只能靠人眼验收的脚本」变成了「可被自动化拦截的构件」。这个决定的价值在后续引入 LLM 时会进一步放大：模型输出不确定，所以必须靠不变量式的测试（是否调用了正确工具、是否在预算内终止）来约束。
4. **AI 协作的正确姿态是「审查 + 验证 + 修改」，而非「接受 + 运行」。** 本实验中最真实的一课是：AI 生成的 `Agent` 与 `Main` 结构上完全正确，却因为一个**空文件**而整体不可编译（§8 问题 1）；AI 生成的测试断言顺序正确、命名合理，却漏掉了 `null` 与大小写分支（§5.3）。两处缺口都不是靠「让 AI 再检查一遍」发现的，而是靠**跑构建**和**人工比对分支表**发现的。
5. **版本控制是「可回退的安全感」。** 有了 `a9464e7 bootstrap project` 这个检查点，后续任何 AI 大范围改写都可以随时对比与回退； `.gitignore` 让仓库只保存「人写的、需要被 review 的」内容，而不保存机器的中间产物。这就是从「写代码」转向「管理软件项目」最直观的一步——你管理的不只是文件，而是**可复现的构建、可回退的历史、可执行的规格**。

附录 A：常用命令速查

```
# 构建与测试
mvn clean test                    # 清理 + 编译 + 单元测试
mvn -q -DskipTests compile        # 只编译（静默）
mvn -DskipTests package           # 打包出 target/agent-cli-0.1.0.jar

# 运行
mvn -q exec:java                                        # 推荐：exec-maven-plugin
java -cp target/classes com.agent.Main                  # classpath 方式
java -jar target/agent-cli-0.1.0.jar                    # 可执行 jar
"hello" | java -cp target/classes com.agent.Main        # Windows PowerShell 管道
echo "hello" | java -cp target/classes com.agent.Main   # macOS / Linux 管道

# 版本控制
git log --oneline
git status
git check-ignore -v target
```

附录 B：验收清单

```
V1
├── Maven Project  ✅  com.agent:agent-cli:0.1.0，release 17，JUnit 5.10.2 (test)
├── Java CLI       ✅  Agent（逻辑）+ Main（I/O 接线），支持 hello / 空白 / 回显 / quit
├── Test           ✅  52 个用例全通过（Tests run: 52, Failures: 0, Errors: 0）
└── Git            ✅  a9464e7 bootstrap project / d06b5ff 补充边界测试与文档
```